
# A MINI PROJECT REPORT

**ON**

**"The Archivist"**

**SUBMITTED BY**

**Name:** Aditya Kuwar **PRN:** 125B1B097

**Name:** Disha Jagtap **PRN:** 125B1B081

**Name:** Tanuja Kamble **PRN:** 125B1B082

**Under the Guidance**
**of**
**Mrs. MEHZABIN PATHAN**

**DEPARTMENT OF COMPUTER ENGINEERING**
**PIMPRI CHINCHWAD COLLEGE OF ENGINEERING**
**2025-2026**

## CERTIFICATE

This is to certify that, the Mini project entitled "**The Archivist**" is successfully carried out as a mini project and successfully submitted by following students of "PCET's Pimpri Chinchwad College of Engineering, Nigdi, Pune-44". Under the guidance of **,Mrs. Mehzabin Pathan** In the partial fulfilment of the requirements for the First Year B. Tech. Computer Engineering.

**Project Guide Name:**

Mrs. Mehzabin Pathan

**Signature:**

**Group Members**

| | |
|---|---|
| Aditya Kuwar | 125B1B097 |
| Disha Jagtap | 125B1B081 |
| Tanuja Kamble | 125B1B082 |

# Table of Contents

## 1. Introduction

The Library Management System (LMS), branded as "The Archivist", is a full-stack web application designed to digitise and automate the core operations of an institutional library. Built on the Python Flask micro-framework with an SQLite relational database, it provides a seamless browser-based interface for both library administrators and student members.

Modern educational institutions require efficient tools for cataloguing books, tracking loans, computing overdue fines, and managing member accounts. The Archivist addresses these requirements through a role-based web portal that eliminates paper-based records and manual tracking, replacing them with a secure, real-time digital system.

The application exposes both a Jinja2-rendered server-side UI and a comprehensive RESTful JSON API, allowing future integration with mobile applications or third-party systems. It enforces authentication and authorisation at every layer, ensuring that students can only access permitted resources while administrators retain full operational control.

## 2. Problem Statement

Traditional library management in educational institutions suffers from several critical inefficiencies:

- Manual record-keeping is error-prone, leading to lost issue records, incorrect fine calculations, and duplicate book entries.
- Students have no self-service mechanism to check book availability, view their borrowing history, or track upcoming due dates without visiting the library in person.
- Administrators spend excessive time on repetitive clerical tasks such as computing fines, updating availability statuses, and locating circulation records.
- There is no reliable mechanism to control which books are visible to students versus reserved for administrative purposes only.
- Physical registers cannot produce instant reports on active issues, overdue books, or fine summaries.

The Archivist is designed to solve all of the above by providing a centralised, role-aware digital system with real-time data access, automatic fine computation, and a clean self-service interface for students.

## 3. Objectives of the Project

- Develop a secure, role-based web application supporting Admin and Student user types.
- Enable administrators to perform full CRUD (Create, Read, Update, Delete) operations on the book inventory.
- Implement an automated book circulation workflow covering issuance, loan period tracking, return processing, and fine calculation.
- Provide students with a personal dashboard to monitor active loans, due dates, borrowing history, and accrued fines.
- Integrate a book-search facility with visibility control so administrators can hide restricted titles from students.
- Expose a RESTful JSON API (parallel to the UI) for all major operations to enable future integrations.
- Use password hashing (Werkzeug PBKDF2) to store credentials securely; never store plain-text passwords.
- Enforce referential integrity at the database level via foreign keys and CHECK constraints.
- Deliver a responsive, accessible UI using Tailwind CSS with a consistent design language across all pages.

## 4. Software / Hardware Requirements

### 4.1 Software Requirements

| Component | Version / Detail | Purpose |
|---|---|---|
| Python | 3.10+ | Core application runtime |
| Flask | 3.x | Web micro-framework (routing, sessions, templating) |
| Werkzeug | Bundled with Flask | Password hashing (PBKDF2-SHA256) & WSGI utilities |
| SQLite | 3.x (stdlib) | Embedded relational database |
| Jinja2 | Bundled with Flask | Server-side HTML templating engine |
| Tailwind CSS | CDN (3.x) | Utility-first CSS framework |
| C++ | C++11 / C++17 | Core logic implementation and algorithm handling |
| Git | Any | Version control |

| | | |
|---|---|---|
| Web Browser | Chrome / Firefox / Edge | Client interface |
| GCC / G++ Compiler | 11+ | Compilation and execution of C++ programs |

### 4.2 Hardware Requirements

| Component | Specification |
|---|---|
| Processor | Intel Core i3 / AMD Ryzen 3 or equivalent |
| RAM | 4 GB minimum (8 GB recommended) |
| Storage | 500 MB free disk space |
| Operating System | Windows 10 / macOS 12 / Ubuntu 20.04 or later |
| Network | Local network or internet (for CDN assets) |

## 5. Algorithm

### 5.1 User Authentication Algorithm

Algorithm LOGIN:
INPUT: username, password
1. Validate that username and password are non-empty.
2. Query users table WHERE username = input_username.
3. IF no user found OR password hash mismatch THEN
   Flash error; redirect to login page.
4. ELSE clear existing session.
5. Store user_id and role in server-side session.
6. Flash welcome message; redirect to role-appropriate dashboard.

Algorithm SIGNUP:
INPUT: username, password, role
1. Validate non-empty fields; normalise role to 'admin' or 'student'.
2. Query users WHERE username = input_username.
3. IF exists THEN flash duplicate error; redirect to login.
4. Hash password using PBKDF2-SHA256 via Werkzeug.

5. INSERT new user record into database.
6. Flash success; redirect to login.

### 5.2 Book Issuance Algorithm

Algorithm ISSUE_BOOK(user_id, book_id, loan_days, enforce_visibility):

1. Fetch book record WHERE id = book_id.
2. IF book not found THEN return error 'Book not found.'
3. IF enforce_visibility AND book.visibility != 1 THEN
   return error 'Book is hidden from student access.'
4. IF book.status != 'Available' THEN
   return error 'Book is not available for issue.'
5. Compute issue_date = TODAY().
6. Compute due_date = issue_date + loan_days (7 or 14 days).
7. INSERT into issued_books (user_id, book_id, issue_date, due_date, fine=0).
8. UPDATE books SET status = 'Issued' WHERE id = book_id.
9. COMMIT transaction.
10. Return success message.

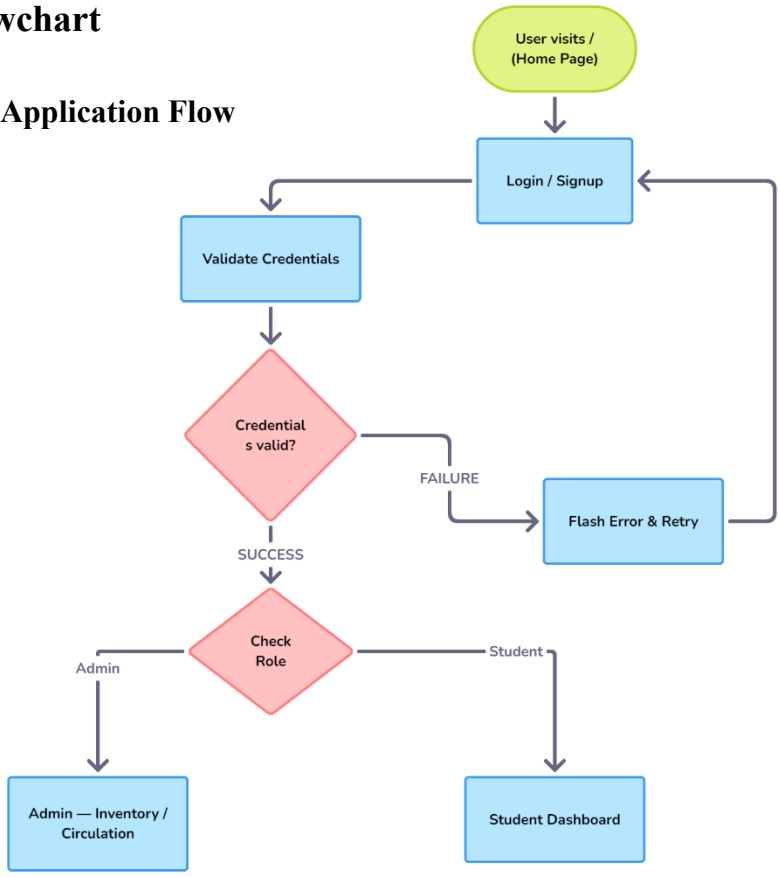
### 5.3 Book Return & Fine Calculation Algorithm

Algorithm RETURN_BOOK(issue_id):

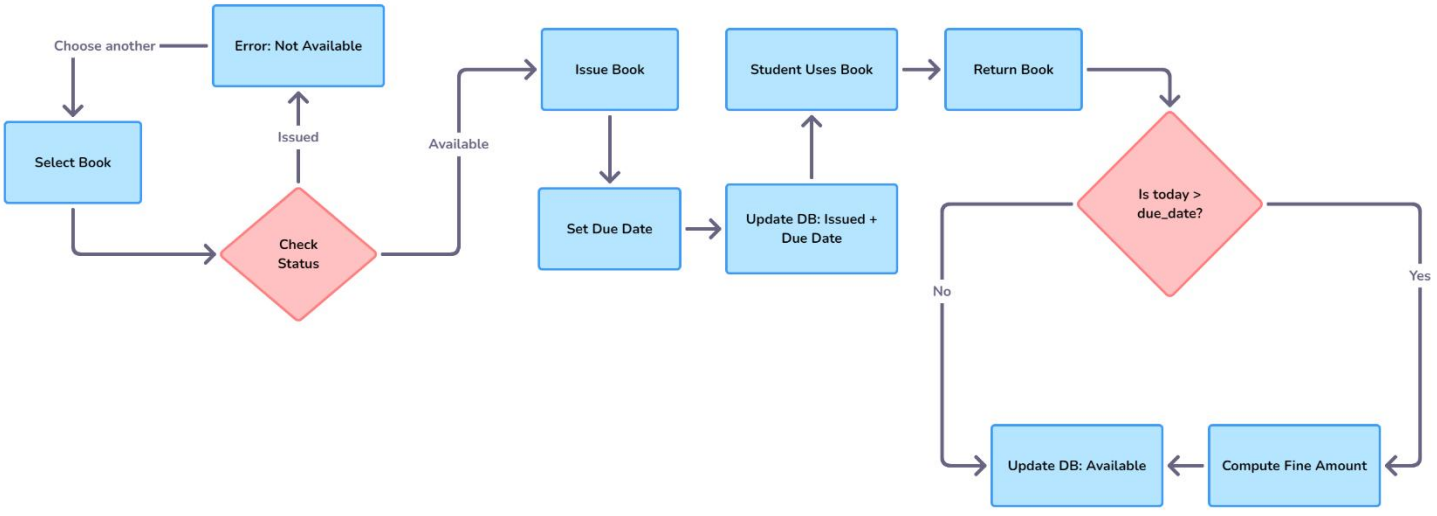
1. Fetch issue record JOIN books WHERE id = issue_id.
2. IF record not found THEN return error.
3. IF return_date IS NOT NULL THEN return error 'Already returned.'
4. Compute today = TODAY().
5. Parse due_date from ISO string.
6. late_days = MAX(0, (today - due_date).days).
7. fine = late_days * FINE_PER_DAY  (Rs. 1.00 per day).
8. UPDATE issued_books SET return_date = today, fine = fine.
9. UPDATE books SET status = 'Available'.
10. COMMIT transaction.
11. Return success with fine amount.

# 6. Flowchart

## 6.1 Overall Application Flow




## 6.2 Circulation Workflow



## 7. Program Code

### 7.1 Database Schema (schema.sql)

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS users (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   username TEXT NOT NULL UNIQUE,
   password TEXT NOT NULL,
   role TEXT NOT NULL CHECK (role IN ('admin', 'student'))
);

CREATE TABLE IF NOT EXISTS books (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT NOT NULL,
   author TEXT NOT NULL,
   status TEXT NOT NULL DEFAULT 'Available'
      CHECK (status IN ('Available', 'Issued')),
   visibility INTEGER NOT NULL DEFAULT 1
      CHECK (visibility IN (0, 1))
);

CREATE TABLE IF NOT EXISTS issued_books (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   user_id INTEGER NOT NULL,
   book_id INTEGER NOT NULL,
   issue_date TEXT NOT NULL,
   due_date   TEXT NOT NULL,
   return_date TEXT,
   fine REAL NOT NULL DEFAULT 0,
   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
   FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE CASCADE
);
```

### 7.2 Flask Application Entry & Configuration (app.py)

```
app = Flask(__name__)
app.config.update(
   SECRET_KEY=os.environ.get('SECRET_KEY', 'change-me-in-production'),
   DATABASE=os.path.join(app.root_path, 'library.db'),
   DEFAULT_LOAN_DAYS=14,
   FINE_PER_DAY=1.0,
)
```

```
def get_db():
    if 'db' not in g:
        g.db = sqlite3.connect(app.config['DATABASE'])
        g.db.row_factory = sqlite3.Row
        g.db.execute('PRAGMA foreign_keys = ON')
    return g.db
```

**7.3 Role-Based Access Control Decorators**

```
def login_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if current_user() is None:
            flash('Please login first.', 'error')
            return redirect(url_for('login'))
        return view(*args, **kwargs)
    return wrapped

def role_required(*roles):
    def decorator(view):
        @wraps(view)
        def wrapped(*args, **kwargs):
            user = current_user()
            if user is None:
                flash('Please login first.', 'error')
                return redirect(url_for('login'))
            if user['role'] not in roles:
                flash('You do not have permission.', 'error')
                return redirect(url_for('home'))
            return view(*args, **kwargs)
        return wrapped
    return decorator
```

### 7.4 Fine Calculation Logic

```
int late_days = 0;

if (return_days > due_days) {
   late_days = return_days - due_days;
}

double fine_per_day = 5.0;

double fine = late_days * fine_per_day;

cout << "Late Days : " << late_days << endl;
cout << "Fine Amount : Rs. " << fine << endl;
```

## 8. Sample Input and Output

### 8.1 User Login

| | Input | Output |
|---|---|---|
| Admin Login | username: admin password: admin123 | Redirect to Admin Inventory page Flash: 'Welcome back, admin!' |
| Student Login | username: student password: student123 | Redirect to Student Dashboard Flash: 'Welcome back, student!' |
| Invalid Login | username: xyz password: wrong | Flash error: 'Invalid username or password.' Stay on login page |

Figure 1: Homepage of the Archivist Library Management System

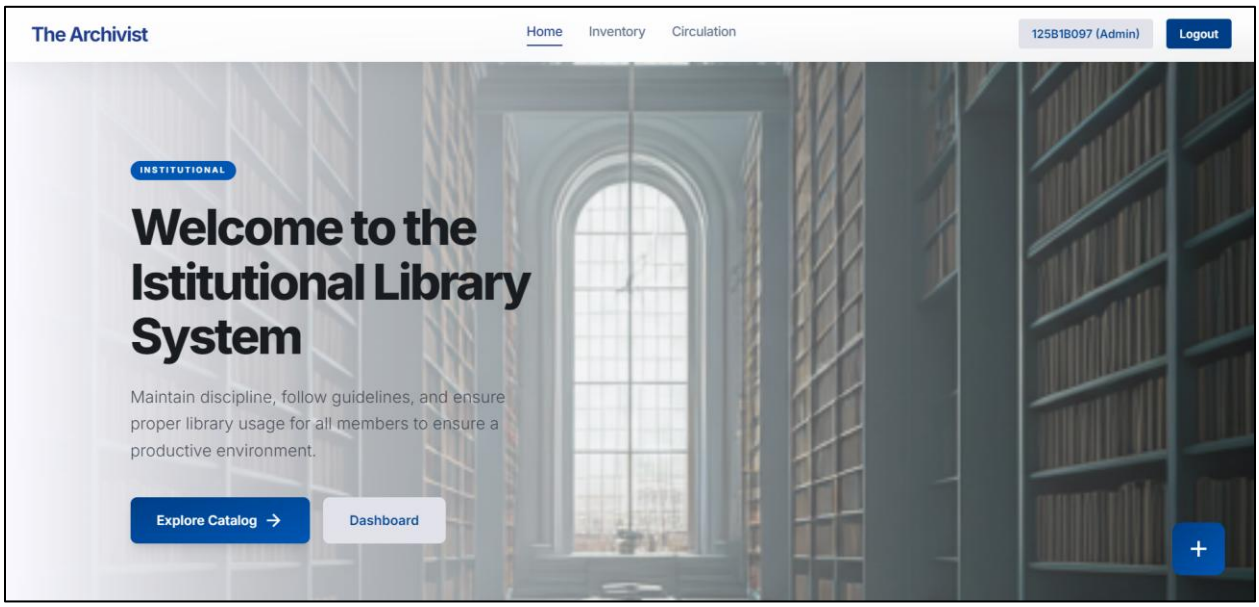
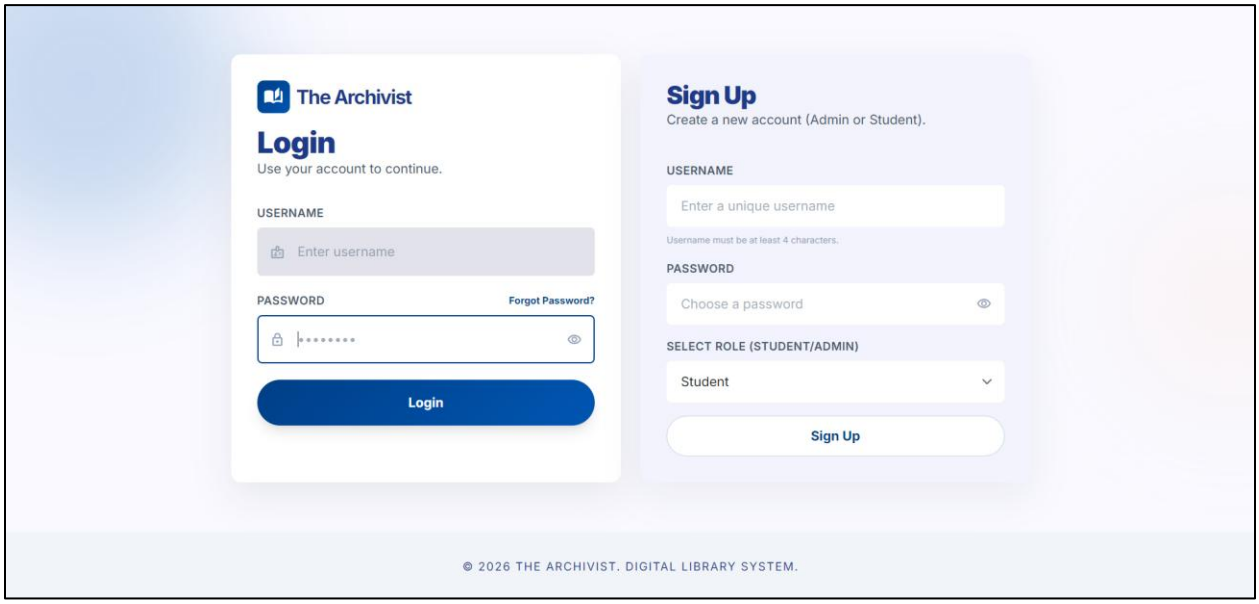



Figure 2: User Login Page of the Archivist Library Management System



### 8.2 Book Issuance

| Scenario | Input | Output |
|---|---|---|
| Successful Issue | book_id: 3, loan_days: 14 | issued_books record created book.status → 'Issued' due_date = today + 14 days |
| Already Issued | book_id: 3 (status=Issued) | Flash: 'Book is not available for issue.' |
| Hidden Book (Student) | book_id: 5 (visibility=0) | Flash: 'Book is currently hidden from student access.' |

Figure 3: Library Circulation Management Interface

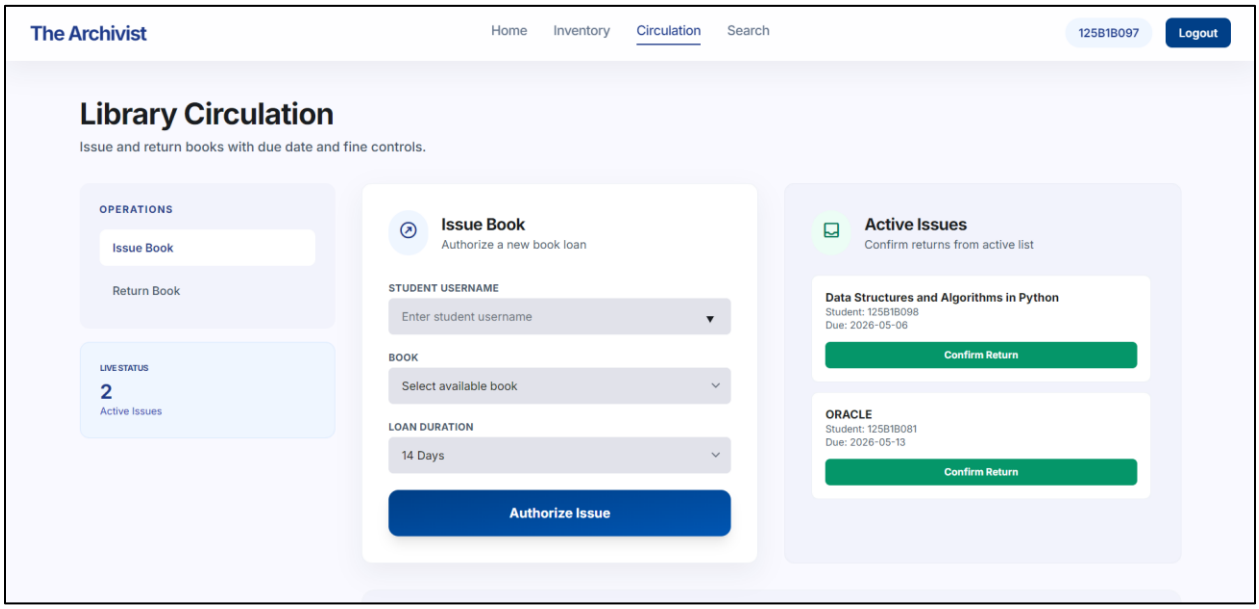



Figure 4: Book Records Management Page

### 8.3 Book Return & Fine

| Scenario | Condition | Output |
|---|---|---|
| On-time Return | return on or before due_date | fine = Rs. 0.00 book.status → 'Available' |
| 3-Day Late Return | return 3 days after due_date | fine = Rs. 3.00 Flash: 'Returned ... Fine: Rs. 3.00' |
| Already Returned | return_date IS NOT NULL | Flash: 'Book has already been returned.' |

Figure 5: Issued Books Overview Dashboard

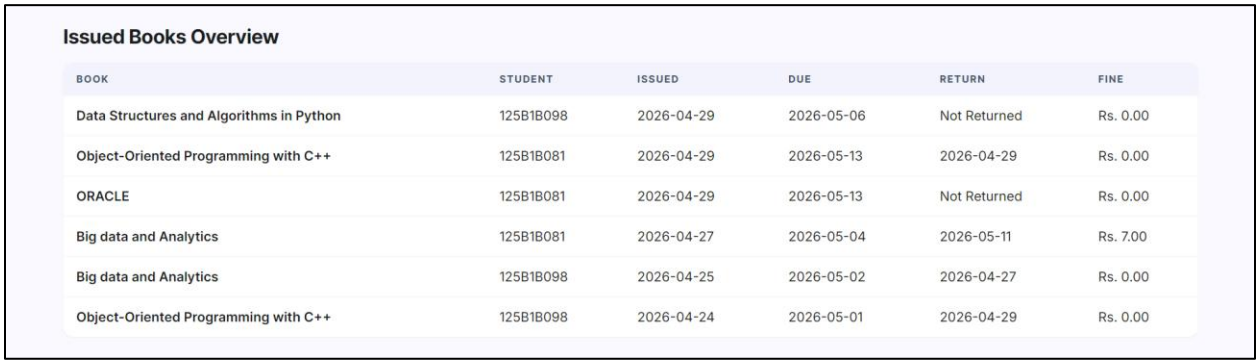
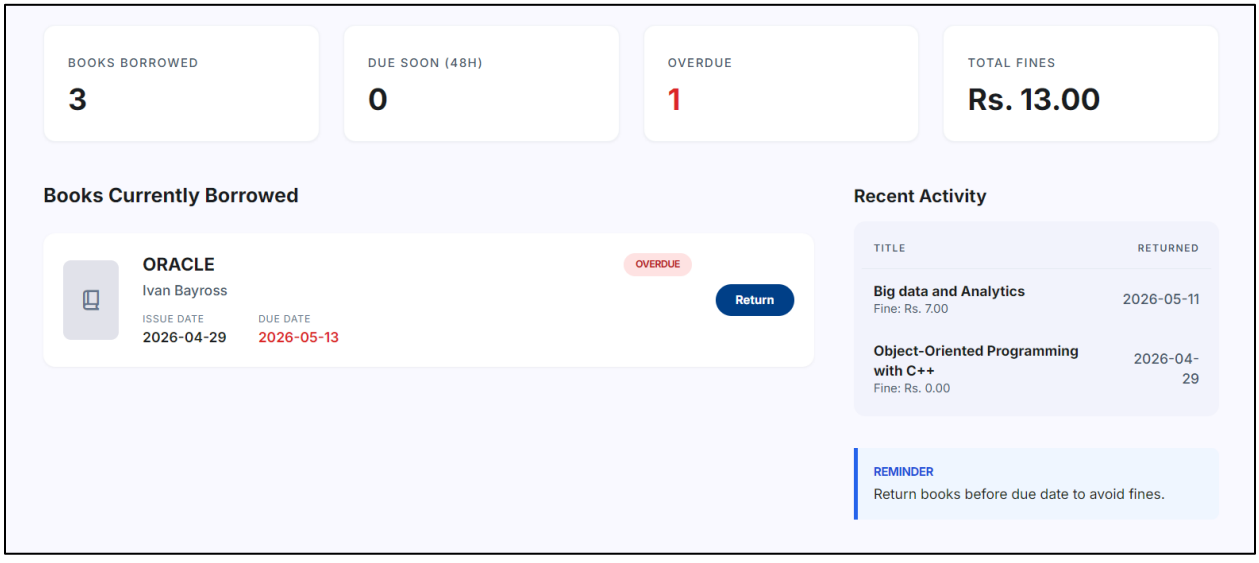



Figure 6: Book Status and Fine Management Dashboard



## 9. Result and Observations

Upon development and testing of The Archivist Library Management System, the following results were observed:

- Authentication: Secure login and signup function correctly. Werkzeug's PBKDF2-SHA256 hashing ensures no plain-text passwords are stored. Invalid credentials are rejected with informative flash messages.
- Role-Based Access: The role_required and login_required decorators correctly restrict page access. Students attempting to access admin URLs are redirected with a permission error.
- Book CRUD: Administrators can add, edit, delete, and toggle visibility of books. Deletion is correctly blocked when a book is currently issued, preventing orphaned records.
- Issuance & Return: The circulation workflow correctly updates book status between 'Available' and 'Issued'. Loan periods of 7 or 14 days are enforced; invalid values default to 14 days.
- Fine Calculation: Fines of Rs. 1.00 per day are automatically computed on return. The student dashboard also displays dynamic accruing fines for unreturned overdue books.
- Student Dashboard: Statistics for total books borrowed, books due within 2 days, overdue books, and total fines are accurately computed via SQL aggregate queries.
- REST API: All API endpoints return correct JSON responses and HTTP status codes (200, 201, 400, 401, 403, 404, 409), enabling programmatic integration.
- Database Integrity: Foreign key constraints and CHECK constraints prevent invalid data entry (e.g., invalid roles, statuses, or visibility values) at the database level.

Overall, the system met all defined objectives during testing. No critical bugs or data inconsistencies were observed in normal operation.

## 10. Future Scope

Upon successful implementation of *The Archivist Library Management System*, several enhancements can be considered in the future to improve functionality, scalability, and user experience. The following features may be added in future versions of the system:

• Integration of barcode or QR code scanning for faster book issue and return operations.
• Addition of email notifications for due dates, overdue books, and password recovery.
• Development of a mobile-friendly application for students and administrators.
• Implementation of online book reservation and renewal facilities.
• Integration of digital resources such as e-books and online journals.
• Enhancement of security through two-factor authentication and audit logging.

These improvements would make the system more efficient, secure, and suitable for deployment in larger educational institutions and modern library environments.

## 11. Learning Outcome

• Learned full-stack web development using Python, Flask, SQLite, HTML, and Tailwind CSS.

• Improved understanding of C++ concepts such as classes, functions, loops, and file handling.

• Learned Flask routing, sessions, flash messages, and decorators.

• Understood database design using tables, primary keys, foreign keys, and SQL queries.

• Implemented security features like password hashing and role-based access control.

• Learned REST API basics using GET, POST, PUT, and DELETE methods.

• Improved problem-solving skills by implementing book issue, return, and fine calculation features.

## 12. Conclusion

The Archivist Library Management System successfully demonstrates a secure and efficient full-stack solution for institutional library management. The system automates book issuance, returns, fine calculation, and inventory handling, replacing manual record-keeping with a role-based digital platform. Its dual-interface architecture, secure authentication, and reliable database design ensure scalability, usability, and data consistency. Future improvements may include email notifications, book renewal features, PDF receipts, pagination, and cloud deployment with PostgreSQL support.

## 13. References


- Pallets Projects. (2024). Flask Documentation (3.x). https://flask.palletsprojects.com/
- Pallets Projects. (2024). Werkzeug Documentation. https://werkzeug.palletsprojects.com/
- SQLite Consortium. (2024). SQLite Documentation. https://www.sqlite.org/docs.html
- Tailwind Labs. (2024). Tailwind CSS Documentation. https://tailwindcss.com/docs
- Ronacher, A. (2018). Flask Web Development. O'Reilly Media.
- Mozilla Developer Network. (2024). HTTP Status Codes Reference. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
- Python Software Foundation. (2024). Python 3 Documentation. https://docs.python.org/3/
- Grinberg, M. (2018). Flask Mega-Tutorial. https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world